Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3
з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Розробка інтерфейсу користувача»

Виконала:
студентка групи ІМ-51
Резник Валерія Вадимівна
Номер в списку - 26

Перевірив:
Рекечинський Дмитро Олександрович

Київ 2020

Мета: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіант: 27 (26+1)

- Статичний масив обсягом 127 елементів
- “Гумовий” слід - пунктирна лінія чорного кольору

  Прямокутник:
  - Від центру до одного з кутів
  - Чорний контур з рожевим заповненням

  Еліпс:
  - За двома протилежними кутами охоплюючого прямокутника
  - Чорний контур без заповнення

- Позначка поточного типу об’єкту в заголовку вікна

Вихідний код:

main.py

```
import sys
from PySide6.QtCore import Qt, QSize, QPoint
from PySide6.QtGui import QAction, QPainter, QPen, QColor, QPalette,
QIcon, QPixmap, QBrush, QMouseEvent
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QWidget,
    QToolBar,
    QMessageBox,
)
from shapes import Shape, Point, Line, Rectangle, Ellipse

N = 127

def create_icon(shape_name):
    pixmap = QPixmap(32, 32)
    pixmap.fill(QColor("#343333"))

    painter = QPainter(pixmap)

    if shape_name == "Крапка":
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor("black"))
        painter.drawEllipse(QPoint(16, 16), 3, 3)

    elif shape_name == "Лінія":
        painter.setPen(QPen(QColor("black"), 2))
        painter.drawLine(4, 27, 28, 5)

    elif shape_name == "Прямокутник":
        painter.setPen(QPen(QColor("black"), 2))
        painter.setBrush(QBrush(QColor("#EFB3E5")))
        painter.drawRect(4, 7, 24, 18)

    elif shape_name == "Еліпс":
        painter.setPen(QPen(QColor("black"), 2))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(4, 7, 24, 18)

    painter.end()
```

```
    return QIcon(pixmap)

class DrawingArea(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.pcshape = []

        self.current_shape = "Крапка"

        self.x1 = 0
        self.y1 = 0
        self.x2 = 0
        self.y2 = 0

        self.drawing = False

        self.setMinimumSize(600, 400)
        self.setAutoFillBackground(True)
        palette = self.palette()
        palette.setColor(QPalette.ColorRole.Window, QColor("white"))
        self.setPalette(palette)

    def set_shape(self, shape_name):
        self.current_shape = shape_name

    def mousePressEvent(self, event: QMouseEvent):
        if event.button() == Qt.MouseButton.LeftButton:
            self.x1 = int(event.position().x())
            self.y1 = int(event.position().y())

            self.x2 = self.x1
            self.y2 = self.y1

            self.drawing = True

            if self.current_shape == "Крапка":
                self.add_shape(Point(self.x1, self.y1))

                self.drawing = False

            self.update()
```

```
    def mouseMoveEvent(self, event: QMouseEvent):
        if self.drawing:
            self.x2 = int(event.position().x())
            self.y2 = int(event.position().y())
            self.update()

    def mouseReleaseEvent(self, event: QMouseEvent):
        if (event.button() == Qt.MouseButton.LeftButton and
self.drawing):
            self.x2 = int(event.position().x())
            self.y2 = int(event.position().y())

            if self.current_shape == "Лінія":
                shape = Line(self.x1, self.y1, self.x2, self.y2)

            elif self.current_shape == "Прямокутник":
                shape = Rectangle(self.x1, self.y1, self.x2, self.y2)

            elif self.current_shape == "Еліпс":
                shape = Ellipse(self.x1, self.y1, self.x2, self.y2)

            else: shape = None

            if shape is not None:
                self.add_shape(shape)

            self.drawing = False
            self.update()

    def add_shape(self, shape):
        if len(self.pcshape) >= N:
            QMessageBox.warning(self, "Увага", "Досягнуто максимальної
кількості фігур: 127")
            return

        self.pcshape.append(shape)

    def paintEvent(self, event):
        painter = QPainter(self)

        for shape in self.pcshape:
            shape: Shape
```

```
            shape.draw(painter)

        if self.drawing and self.current_shape != "Крапка":

            if self.current_shape == "Лінія":
                preview_shape = Line(self.x1, self.y1, self.x2,
self.y2)

            elif self.current_shape == "Прямокутник":
                preview_shape = Rectangle(self.x1, self.y1, self.x2,
self.y2)

            elif self.current_shape == "Еліпс":
                preview_shape = Ellipse(self.x1, self.y1, self.x2,
self.y2)

            preview_shape.draw_preview(painter, self.x2, self.y2)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Lab3")
        self.resize(800, 600)

        self.drawing_area = DrawingArea(self)
        self.setCentralWidget(self.drawing_area)

        self.shape_actions = {}

        self.create_menu()
        self.create_toolbar()

    def create_menu(self):
        menu_bar = self.menuBar()

        file_menu = menu_bar.addMenu("Файл")
        clear_action = QAction("Очистити", self)
        clear_action.triggered.connect(self.clear_canvas)
        file_menu.addAction(clear_action)

        self.objects_menu = menu_bar.addMenu("Об'єкти")
```

```
        shape_names = ["Крапка", "Лінія", "Прямокутник", "Еліпс"]

        self.shape_actions: dict[str, QAction] = {}

        for name in shape_names:
            action = QAction(create_icon(name), name, self)
            action.setToolTip(name)
            action.triggered.connect(lambda checked=False, n=name:
self.select_shape(n))

            self.objects_menu.addAction(action)
            self.shape_actions[name] = action

        help_menu = menu_bar.addMenu("Довідка")
        about_action = QAction("Про програму", self)
        about_action.triggered.connect(self.show_about)
        help_menu.addAction(about_action)

    def create_toolbar(self):
        toolbar = QToolBar("Інструменти", self)
        toolbar.setMovable(False)
        toolbar.setIconSize(QSize(32,32))

        self.addToolBar(Qt.ToolBarArea.TopToolBarArea, toolbar)

        self.toolbar_actions: dict[str, QAction] = {}

        for name in self.shape_actions:
            action = QAction(create_icon(name), name, self)
            action.setCheckable(True)
            action.setToolTip(name)
            action.triggered.connect(lambda checked=False, n=name:
self.select_shape(n))

            toolbar.addAction(action)
            self.toolbar_actions[name] = action

        self.toolbar_actions["Крапка"].setChecked(True)

    def select_shape(self, shape_name):
        self.drawing_area.set_shape(shape_name)

        for name, action in self.toolbar_actions.items():
```

```
            action.setChecked(name == shape_name)

        self.setWindowTitle(f"Lab3 - {shape_name}")

    def clear_canvas(self):
        self.drawing_area.pcshape.clear()
        self.drawing_area.update()

    def show_about(self):
        QMessageBox.about(self, "Про програму", "Lab3 - графічний
редактор\n")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()

    sys.exit(app.exec())
```

shapes.py

```
from abc import ABC, abstractmethod
from PySide6.QtCore import Qt
from PySide6.QtCore import QPoint
from PySide6.QtGui import QColor, QPainter, QPen, QBrush

class Shape(ABC):
    def __init__(self, x1: int, y1: int, x2: int, y2: int):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    @abstractmethod
    def draw(self, painter: QPainter):
        pass

    @abstractmethod
    def draw_preview(self, painter: QPainter, x2: int, y2: int):
        pass

class Point(Shape):
```

```
    def __init__(self, x: int, y: int):
        super().__init__(x, y, x, y)

    def draw(self, painter: QPainter):
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor("black"))
        painter.drawEllipse(QPoint(self.x1, self.y1), 3, 3)

    def draw_preview(self, painter: QPainter, x2: int, y2: int):
        pass

class Line(Shape):
    def draw(self, painter: QPainter):
        painter.setPen(QPen(QColor("black"), 3))
        painter.drawLine(self.x1, self.y1, self.x2, self.y2)

    def draw_preview(self, painter: QPainter, x2: int, y2: int):
        painter.setPen(QPen(QColor("black"), 3, Qt.PenStyle.DashLine))
        painter.drawLine(self.x1, self.y1, x2, y2)

class Rectangle(Shape):
    def draw(self, painter: QPainter):
        radius_x = abs(self.x2 - self.x1)
        radius_y = abs(self.y2 - self.y1)

        left = self.x1 - radius_x
        top = self.y1 - radius_y
        width = radius_x * 2
        height = radius_y * 2

        painter.setPen(QPen(QColor("black"), 3))
        painter.setBrush(QBrush(QColor("#EFB3E5")))
        painter.drawRect(left, top, width, height)

    def draw_preview(self, painter: QPainter, x2: int, y2: int):
        radius_x = abs(x2 - self.x1)
        radius_y = abs(y2 - self.y1)

        left = self.x1 - radius_x
        top = self.y1 - radius_y
        width = radius_x * 2
        height = radius_y * 2
```

```
        painter.setPen(QPen(QColor("black"), 3, Qt.PenStyle.DashLine))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawRect(left, top, width, height)

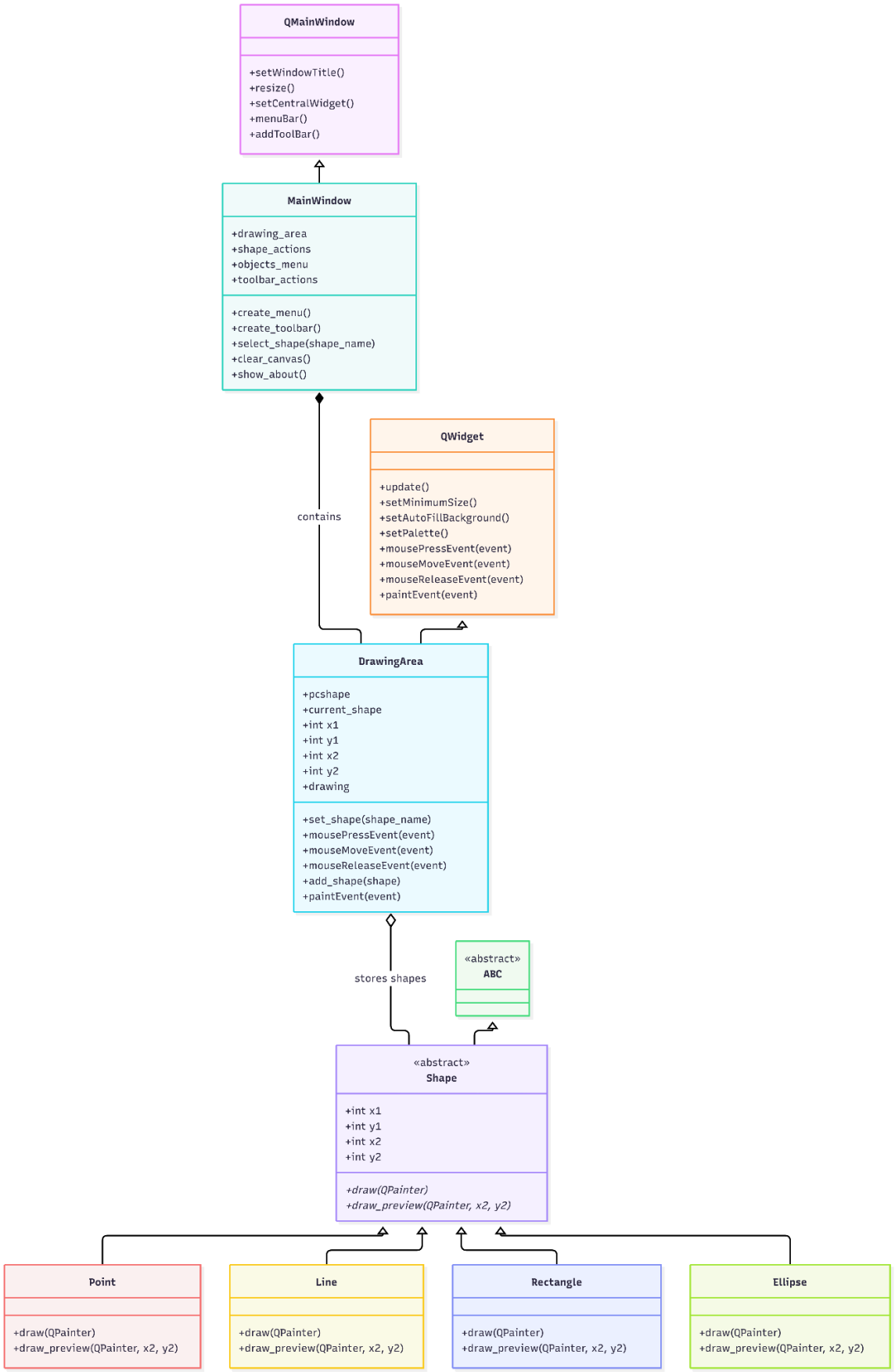
class Ellipse(Shape):
    def draw(self, painter: QPainter):
        left = min(self.x1, self.x2)
        top = min(self.y1, self.y2)
        width = abs(self.x2 - self.x1)
        height = abs(self.y2 - self.y1)

        painter.setPen(QPen(QColor("black"), 3))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(left, top, width, height)

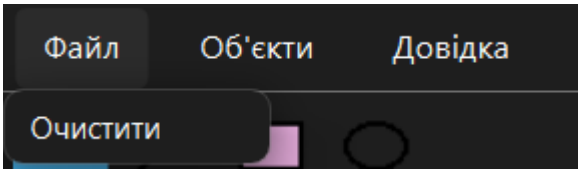
    def draw_preview(self, painter: QPainter, x2: int, y2: int):
        left = min(self.x1, x2)
        top = min(self.y1, y2)
        width = abs(x2 - self.x1)
        height = abs(y2 - self.y1)

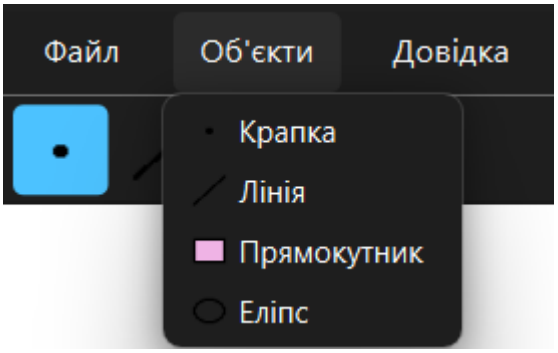
        painter.setPen(QPen(QColor("black"), 3, Qt.PenStyle.DashLine))
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(left, top, width, height)
```

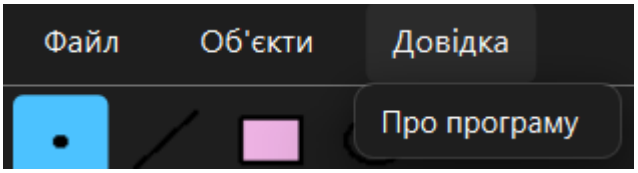
Схема успадкування класів:



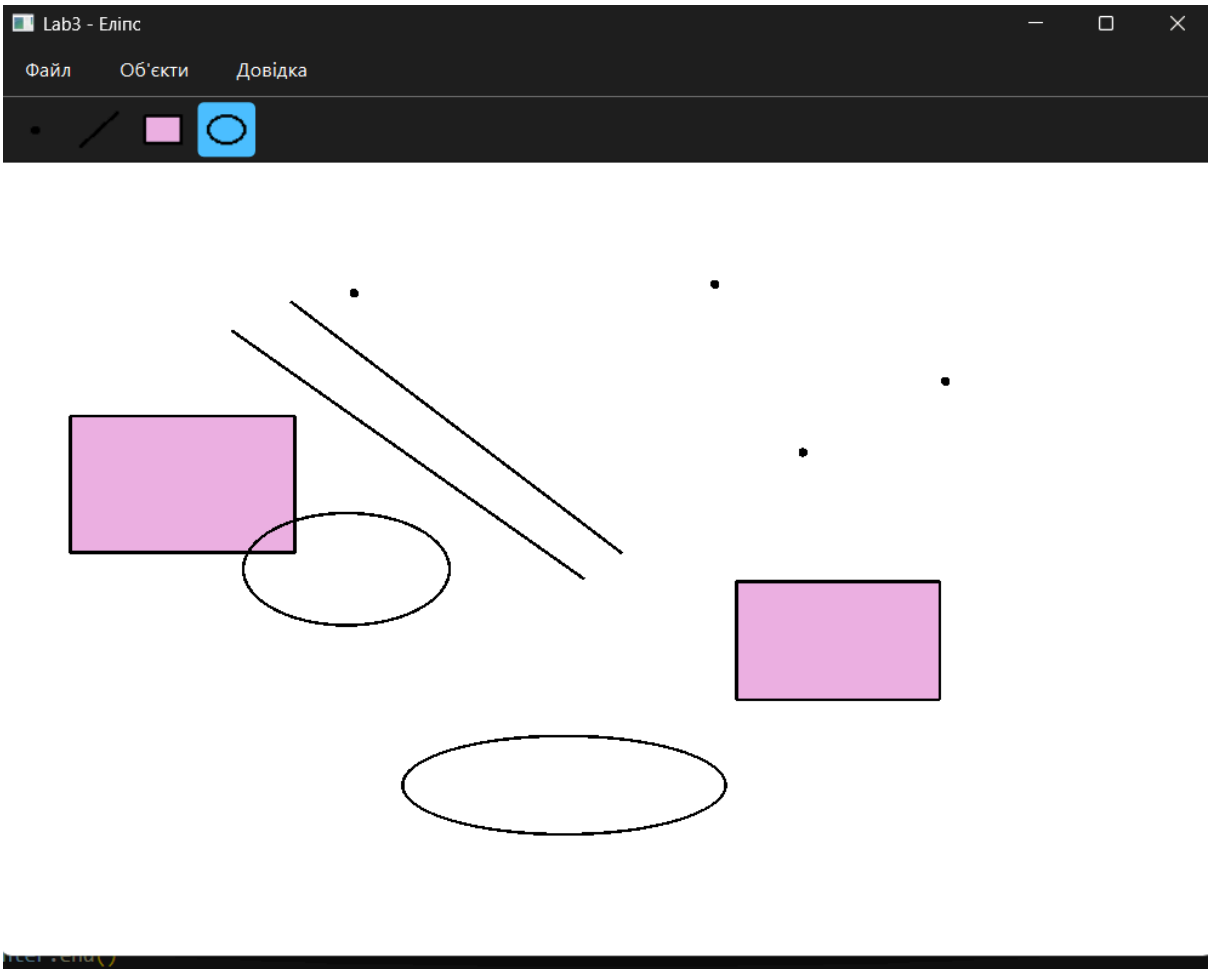
Тестування (скріншоти):











Висновок:

Під час виконання лабораторної роботи було створено графічний редактор із використанням бібліотеки PySide6. Було закріплено навички застосування інкапсуляції, абстракції, успадкування та поліморфізму. Реалізовано класи для різних геометричних фігур, динамічний список для їх зберігання, меню та панель інструментів для вибору фігур. У результаті роботи отримано практичні навички об'єктно-орієнтованого програмування та створення графічних інтерфейсів користувача.